



**Building a better
working world**

Ernst & Young (EY)

Summer Internship

Project Title: Generative AI Integration

Duration: Sept 2024 - Nov 2024

Introduction

EY has been at the forefront of innovation and professional services for decades, combining advanced technology with strategic insights. Based in London, England, the company is renowned for its commitment to fostering a culture of innovation and excellence. I chose my internship at EY because of its reputation as a one of the Big Four accounting firms and its impact on global clients through cutting-edge technology and strategic consulting.

Problem Statement

In the field of Generative AI, Document Intelligence, and Retrieval-Augmented Generation (RAG), effectively managing and processing large volumes of unstructured text data remains a significant challenge. Existing approaches often struggle with efficiently extracting, embedding, storing, and retrieving relevant information from diverse document sources while maintaining computational efficiency, accuracy, and data security. This project aims to give a understanding of modern methods and convention to develop a streamlined solution that integrates advanced document processing techniques with retrieval-based generative models, ensuring fast, contextually accurate responses for various use cases.

Internship Description

During my internship at EY, I worked on developing and integrating Generative AI models from Hugging Face to address real-world applications. My responsibilities included exploring modern methodologies, identifying potential use cases for generative models, and conducting in-depth research, all while understanding the importance of handling clients' sensitive data. A significant part of my role involved testing these models for accuracy and efficiency, both locally and through online methods. Additionally, I audited the Transformer Model, detailed in the 2017 paper [Attention Is All You Need](#) and focused closely on Meta's Llama 3.1 (8B), providing comprehensive insights into their architectures and performance.

Findings & Discussion

Throughout my internship, I gained valuable insights into the field of AI and what it entails to work at an industry level. I developed key skills in Generative AI, Retrieval-Augmented Generation (RAG), managing Python dependencies, and maintaining ethical AI practices. I executed Llama 3.1 locally and explored multiple embedding models from the [Massive Text Embedding Benchmark \(MTEB\)](#) in conjunction with the Chroma vector database. This hands-on experience deepened my understanding of generative AI and its broader ecosystem.

Progress Update 1

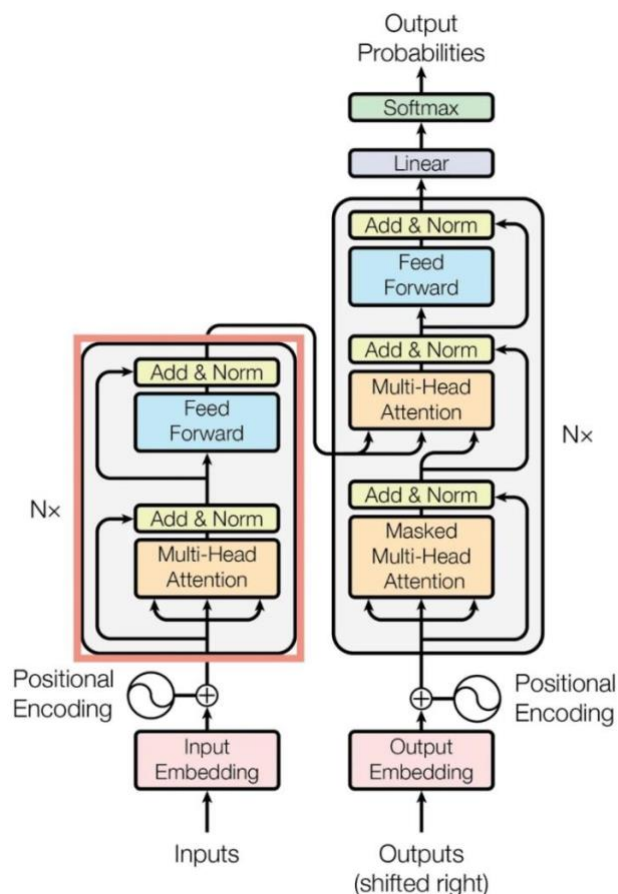
11th September 2024

Topics Covered:

- Brief overview of the Transformer Model (2017)
- Overview on Meta's Llama 3.1
- Takeaways from the [Open Source AI Is the Path Forward](#)

1. Brief overview of the Transformer Model (2017)

Transformers are a revolutionary deep learning architecture introduced in the paper “Attention is All You Need” in 2017. They have since become the foundation for various natural language processing (NLP) tasks and LLMs.



Working of the Transformer Model according to “Attention is All you Need”

Embeddings: To help the Transformer understand **words**, we **break them down into smaller parts** and give each part a **special number**. It's like making a code for each word. This helps the **Transformer see the similarities and differences between words**, just like you recognize patterns in different stories.

Attention Mechanism: Transformers use attention mechanism that allows the model to weigh the importance of different input tokens when processing a specific token. An attention mechanism is a computational approach that forces the model to focus on specific parts of input sequence.

Self-Attention

- **Core concept:** Analyzes how each element in a sequence relates to all other elements.
- **Process:** Each element in the sequence is transformed into three vectors: Query (Q), Key (K), and Value (V).
- A compatibility score is calculated between each pair of elements using the Q and K vectors.
- These scores are normalized using softmax to create attention weights, indicating how much “attention” each element should pay to others.
- The attention weights are used to weight the V vectors, essentially creating a context-aware representation for each element based on its relationship with others.

Multi-Head Self-Attention

- **Builds upon self-attention:** Performs multiple self-attention operations in parallel, with each operation learning to focus on different aspects of the relationships between elements.
- **Process (similar to self-attention but in parallel):**
 - The input is projected into multiple sets of Q, K, and V vectors for each “head.”
 - Separate attention scores and weighted outputs are calculated for each head.
 - The outputs from all heads are concatenated to form the final output.

Analogy to understand the above terms better

Imagine you’re reading a sentence. Self-attention would be like considering how each word relates to every other word to understand the overall meaning. Multi-head selfattention would be like reading the sentence several times, each time focusing on a different aspect like grammar, word relationships, or sentiment. By combining these focused readings, you get a richer understanding of the sentence.

2. Overview on Meta’s Llama 3.1

- Open Source Model available in 3 variants: 405B , 70B, 8B • Ways to run 405B: Meta AI, Groq, NVIDIA AI Foundry • Decoder-only transformer architecture.
- Llama 3 was released on April 18, 2024 with only 8B and 70B variants.
- Llama 3.1 released on July 23, 2024 with 405B variant and updated 8B and 70B variants.
- Llama 3 uses a tokenizer with a vocabulary of 128k tokens that encodes language much more efficiently.

- Compared to the 2017 Attention model, self-attention is replaced with **group multi-query attention** to save attention compute in Llama 2 while also being a decoder only model.

Tokenizer: A tokenizer is a component in natural language processing (NLP) models that breaks down text into smaller units called tokens. Tokens can be words, subwords, characters, or even parts of words. Tokenizers convert text into a format that the model can understand and process.

Vocabulary of 128k Tokens: The "vocabulary" refers to the set of all tokens that the tokenizer can recognize and process. In Llama 3, this vocabulary contains 128,000 unique tokens. This large vocabulary allows the model to represent language more flexibly and accurately. It can handle a wide range of words, subwords, and special characters that appear in different languages, dialects, and technical terms.

Implements Byte Pair Encoding (BPE) to convert text into tokens and then convert back to text without any loss. To scale training for the models, 4D parallelism was used which represents:

- (a) Tensor Parallelism
- (b) Pipeline Parallelism
- (c) Context Parallelism
- (d) Data Parallelism

3. Takeaways from the *Open Source AI Is the Path Forward* - Mark Zuckerberg, Founder and CEO of Meta

The blog compares the development of open-source AI to the evolution of open-source Linux in recent years, which has become the industry standard for cloud computing and the operating systems powering most mobile devices.

Many companies of today develop leading closed models and that open source models are slowly catching up to them. In the coming years, it is expected that future Llama models will become the most advanced in the industry. But even before that, Llama is already leading on openness, modifiability, and cost efficiency.

Llama 3 aims to be the open-source AI's leading industry standard. Meta is working with a range of companies to grow the broad ecosystem. Amazon, Databricks, and NVIDIA are launching full suites of services to support developers fine-tuning and distilling their own models. The models will be available on all major clouds including AWS, Azure, Google, Oracle, and more.

Why Open Source AI is Good for Developers

1. **Customization and Control:** Developers can train, fine-tune, and distill models to fit specific needs and data without vendor lock-in.
2. **Data Security:** Open-source models allow organizations to run AI on their own infrastructure, reducing risks associated with data sharing with closed model providers.
3. **Efficiency and Cost-Effectiveness:** Running inference on open-source models like Llama is more affordable than using proprietary models.
4. **Long-Term Investment:** Open-source ecosystems are advancing rapidly, providing a future-proof foundation for developers.

Why Open-Source AI is Good for Meta

1. **Avoiding Ecosystem Lock-in:** Meta avoids dependency on closed ecosystems, similar to challenges faced with Apple's restrictive environment.
2. **Building an Ecosystem:** By open-sourcing Llama, Meta aims to foster a robust ecosystem of tools and integrations, ensuring competitiveness and innovation.
3. **Different Business Model:** Unlike closed AI providers, Meta's revenue does not depend on selling model access, allowing it to openly release AI models without impacting its business.
4. **Proven Track Record:** Meta has successfully used open-source strategies before with projects like PyTorch, React, and the Open Compute Project.

Future Vision and Call to Action

Meta aims to build a strong ecosystem around Llama by partnering with developers and companies to expand the functionality of open-source AI. Meta envisions Llama 3.1 as a turning point, leading to widespread adoption of open-source AI and inviting global participation to drive AI benefits universally.

Topics Covered:

- Introduction to Hugging Face
- Implement Llama 3.1 (8B model) locally
- Learning Takeaways from the discussion

1. Introduction to Hugging Face

The Hugging Face Hub is a platform with over 900k models, 200k datasets, and 300k demo apps (Spaces), all open source and publicly available, in an online platform where people can easily collaborate and build ML together. The Hub works as a central place where anyone can explore, experiment, collaborate, and build technology with Machine Learning.

2. Implement Llama 3.1 (8B model) locally

Transformers: Hugging Face Transformers is an open-source Python library that provides access to thousands of pre-trained Transformers models for natural language processing (NLP), computer vision, audio tasks, and more. It simplifies the process of implementing Transformer models by abstracting away the complexity of training or deploying models in lower level ML frameworks like PyTorch and TensorFlow.

Accelerate: this library to help users easily train a Transformers model on any type of distributed setup, whether it is multiple GPU's on one machine or multiple GPU's across several machines.

Bitsandbytes: bitsandbytes enables accessible large language models via k-bit quantization for PyTorch. bitsandbytes provides three main features for dramatically reducing memory consumption for inference and training:

- 8-bit optimizers use block-wise quantization to maintain 32-bit performance at a small fraction of the memory cost.
- QLoRA (Quantized Low-Rank Adaptation) or 4-bit quantization enables large language model training with several memory-saving techniques that don't compromise performance. This method quantizes a model to 4-bits and inserts a small set of trainable low-rank adaptation (LoRa) weights to allow training.

Quantization: Quantization is a technique to reduce the computational and memory costs of running inference by representing the weights and activations with lowprecision data types like 8-bit integer (int8) instead of the usual 32-bit floating point (float32).

Reducing the number of bits means the resulting model requires less memory storage, consumes less energy (in theory), and operations like matrix multiplication can be

performed much faster with integer arithmetic. It also allows to run models on embedded devices, which sometimes only support integer data types.

```
import transformers
import torch

model_id = "meta-llama/Meta-Llama-3.1-8B-Instruct"

#Load Model with 4 bit quantization
model = transformers.AutoModelForCausalLM.from_pretrained(
    model_id,
    device_map="auto",
    torch_dtype=torch.bfloat16,
    quantization_config=transformers.BitsAndBytesConfig(
        load_in_4bit=True,
        bnb_4bit_compute_dtype=torch.bfloat16,
        bnb_4bit_use_double_quant=True,
        bnb_4bit_quant_type='nf4'
    )
)
```

4-bit quantization for Llama 3.1 (8B) on Google Colab

- **AutoModelForCausalLM:** This is a Hugging Face class used to load models that can generate text (language models).
- **device_map="auto":** Automatically selects the best device (CPU or GPU) for running the model based on available hardware.
- **torch_dtype=torch.bfloat16:** Specifies that the model should use the bfloat16 data type, which is a compact, efficient format for reducing memory usage.
- **quantization_config:** This section sets up the 4-bit quantization to further reduce the model's memory usage:
 - **load_in_4bit=True:** Loads the model in 4-bit precision.
 - **bnb_4bit_compute_dtype=torch.bfloat16:** Uses bfloat16 for computations during inference.
 - **bnb_4bit_use_double_quant=True:** Enables double quantization, which helps maintain precision.
 - **bnb_4bit_quant_type='nf4':** Uses a specific quantization type called NF4 (normal-float 4-bit), which is more accurate than regular 4-bit quantization.


```
tokenizer = transformers.AutoTokenizer.from_pretrained(model_id)

pipeline = transformers.pipeline(
    "text-generation",
    model=model,
    tokenizer=tokenizer
)
```

Using the tokenizer from the pre-trained model

- **tokenizer = transformers.AutoTokenizer.from_pretrained(model_id):** This line loads a pre-trained tokenizer associated with the specified model (which refers to the "meta-llama/Meta-Llama-3.1-8B-Instruct").
- **pipeline = transformers.pipeline("text-generation", model=model, tokenizer=tokenizer):**
- **pipeline:** This is a simplified way to apply common tasks like text generation, translation, or summarization.
- **"text-generation":** Specifies that the task is generating text.

3. Learning Takeaways

- **Setting Hugging Face token correctly:** Earlier in the year, I overlooked several key steps while using Hugging Face, but I have since gained clarity on them.
- **GPU Requirement for Model:** Running transformer models locally requires a GPU for optimal performance. Since MacBooks lack dedicated GPUs, this task was carried out on Google Colab instead.

Progress Update 3

19th September 2024

Topics Covered:

- Comparing time taken after adjusting max_new_tokens.
- Identifying use cases for Llama 3.1 (8B model).

1. Comparing time taken when max_new_tokens is set at: •

256

```
import time

messages = [
    {"role": "system", "content": "You are a programming assistant."},
    {"role": "user", "content": "Show me the implementation of a Linked List in Java"},
]

start_time = time.time()

outputs = pipeline(messages, max_new_tokens=256)

end_time = time.time()

execution_time = end_time - start_time

print(f"Execution started at: {time.strftime('%Y-%m-%d %H:%M:%S', time.localtime(start_time))}")
print(f"Execution ended at: {time.strftime('%Y-%m-%d %H:%M:%S', time.localtime(end_time))}")
print(f"Total execution time: {execution_time:.2f} seconds")
```

Total execution time is **21.71 seconds** and output **stops mid-code**

- 512

```
import time

messages = [
    {"role": "system", "content": "You are a programming assistant."},
    {"role": "user", "content": "Show me the implementation of a Linked List in Java"},
]

start_time = time.time()

outputs = pipeline(messages, max_new_tokens=512)

end_time = time.time()

execution_time = end_time - start_time

print(f"Execution started at: {time.strftime('%Y-%m-%d %H:%M:%S', time.localtime(start_time))}")
print(f"Execution ended at: {time.strftime('%Y-%m-%d %H:%M:%S', time.localtime(end_time))}")
print(f"Total execution time: {execution_time:.2f} seconds")
```

Total execution time is **44.73 seconds**, and output is **complete** but stops during explanation.

- 1024

```
import time

messages = [
    {"role": "system", "content": "You are a programming assistant."},
    {"role": "user", "content": "Show me the implementation of a Linked List in Java"},
]

start_time = time.time()

outputs = pipeline(messages, max_new_tokens=1024)

end_time = time.time()

execution_time = end_time - start_time

print(f"Execution started at: {time.strftime('%Y-%m-%d %H:%M:%S', time.localtime(start_time))}")
print(f"Execution ended at: {time.strftime('%Y-%m-%d %H:%M:%S', time.localtime(end_time))}")
print(f"Total execution time: {execution_time:.2f} seconds")
```

Total execution time is **57.95 seconds**, and output is **complete** with explanation. **2.**

Uses cases of Generative AI (text-generation)

Generative AI has experienced remarkable growth in recent years, revolutionizing various domains such as healthcare, finance, education, and more. Furthermore, the technology's capabilities in generating synthetic data, automating complex tasks, and driving innovative research are transforming industries, making processes more efficient and opening new frontiers for exploration and growth.

Several use cases can be found for Generative AI as follows:

- Content Creation for Marketing
- Copywriting and Ad Creation
- Customer Support and Chatbots
- SEO Content Optimization
- Personalized Communication
- Educational Content and Tutoring
- Summarization of Large Texts
- Script and Story Generation

Progress Update 4

26th September 2024

Topics Covered:

- Understanding Word Embeddings.
- Exploring various embedding models available.
- Extract text from a pdf and embed it using an embedding model.

1. Understanding Word Embeddings:

Document Intelligence refers to the use of AI techniques to extract, understand, and analyse data from documents such as PDFs, scanned images, forms, invoices, contracts, and more. It aims to convert unstructured data into structured, actionable insights.

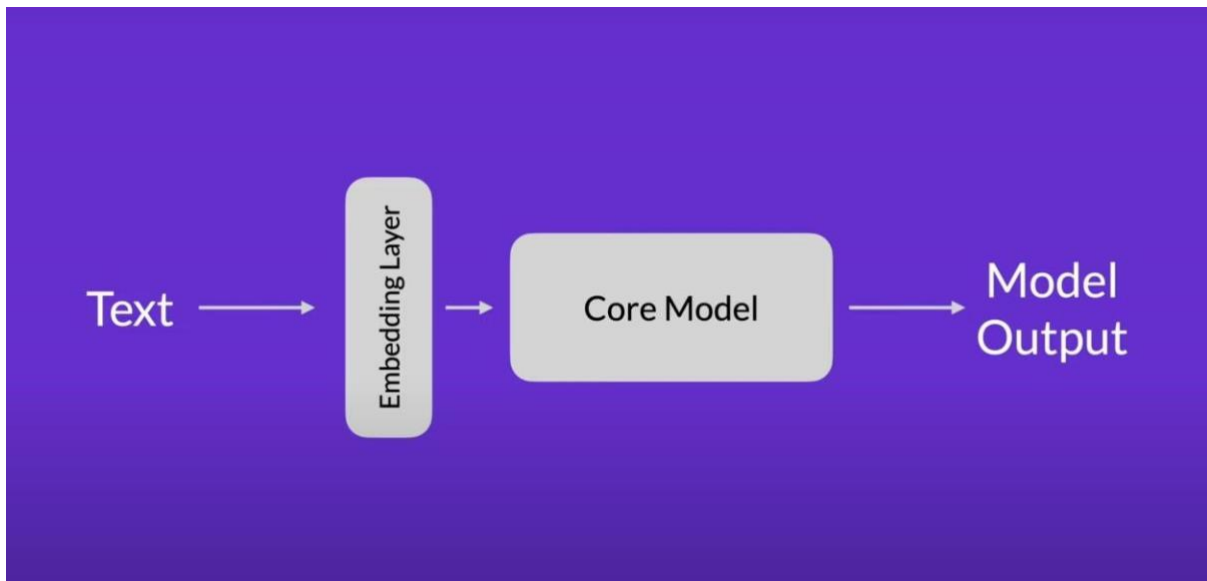
As groundbreaking as LLMs (Large Language Models) may be, they have a few limitations:

- **They're limited by the amount of training data they have access to.**
- **They're generic and lack subject-matter expertise.** LLMs are trained on a large dataset that covers a wide range of topics, but they don't possess specialized knowledge in any field.
- **Citations are tricky.** LLMs don't have a reliable way of returning the exact location of the text where they retrieved the information.

To solve the shortcomings of the LLMs, a technique called **Retrieval Augmented Generation (RAG)** is used that combines *retrieval-based search* with Gen AI models to produce responses or generate text that is informed by a knowledge base or external data sources.

Word Embeddings are mathematical representations of texts. ML models can understand numbers better than actual texts, so texts generally need to be converted into numbers before feeding it into the model. There are several ways to embed your text data which are as follows:

- **One-Hot encoding:** Not most efficient for a large vocabulary
- **Count based approach** (Bag of Words, N-gram, TF-IDF approach): Do not take any context in account, cannot deal with unknown words, embeddings produced are very sparse.
- **Embeddings:** aim to represent the word in a dense vector, while making sure that similar words are close to each other in the embedding space.



Working of an Embedding Model

2. Exploring various embedding models available

This has been done by referring to the followings links:

- [Link 1](#)
- [Link 2](#)

The numerical form is meant to convey the text's semantic relations, context, and sense. **Word embedding** is concerned with the representation of words as fixed dimensional vectors in a specific text. However, **text embedding** involves the conversion of whole text paragraphs, sentences, or documents into numerical vectors.

Popular text embedding models:

- Word2Vec
- GloVe (Global Vectors for word representation)
- FastText
- ELMO (Embeddings from Language Models)
- BERT (Bidirectional Encoder Representations from Transformers)
- GPT (Generative Pre-trained Transformer)
- Doc2Vec
- USE (Universal Sentence Encoder)

When implementing text embedding models, key considerations include-

- Compatibility of the model with the objectives of the application
- Scalability for large datasets
- Interpretability of generated embeddings and
- Resources necessary for effective integration of computational.

3. Extract text from a pdf and embed it using an embedding model

For this task, I explored the following:

- **Gensim** - Gensim is an open-source Python library used for natural language processing (NLP) tasks, especially topic modeling, document similarity, and word embedding. It supports various embedding models like Word2Vec, FastText, and Doc2Vec, allowing efficient training on large text corpora.
- **BERT (Bidirectional Encoder Representations from Transformers)**: BERT is a state-of-the-art transformer-based embedding model developed by Google that captures the context of words in a sentence bidirectionally. Unlike traditional models, which process text in a single direction, BERT reads text in both left-to-right and right-to-left, allowing it to understand the nuances of word meanings based on context.
- **stella_en_1.5B_v5**: The stella_en_1.5B_v5 model is a powerful text embedding model developed by the maintainer **dunzhang**. This model is built on top of the Alibaba-NLP/gte-large-en-v1.5 and Alibaba-NLP/gte-Qwen2-1.5BInstruct models, inheriting their robust natural language processing capabilities.

A key feature of the stella_en_1.5B_v5 model is its support for two main prompts - "s2p_query" for sentence-to-passage tasks and "s2s_query" for sentence-to-sentence tasks. This streamlined approach simplifies the usage of prompts for various NLP applications.

What Are Vector Stores?

Vector stores are databases explicitly designed for efficiently storing and retrieving vector embeddings. They are needed because traditional databases like SQL are not optimized for storing and querying large vector data.

Chroma DB is an open-source vector store used for storing and retrieving vector embeddings. Its main use is to save embeddings along with metadata to be used later by large language models. Additionally, it can also be used for semantic search engines over text data.

Chroma DB features

- Simple and powerful
- Multi-language support
- Open source
- Speed and simplicity

```

import fitz # PyMuPDF
from sentence_transformers import SentenceTransformer

# Load the pre-trained model
model = SentenceTransformer("dunzhang/stella_en_1.5B_v5", trust_remote_code=True).cuda()

# Function to extract text from a PDF
def extract_text_from_pdf(pdf_path):
    text = ""
    with fitz.open(pdf_path) as pdf_document:
        for page_num in range(pdf_document.page_count):
            page = pdf_document.load_page(page_num)
            text += page.get_text()
    return text

pdf_path = "file_name.pdf"

# Extract text from the PDF
extracted_text = extract_text_from_pdf(pdf_path)
print("Extracted Text:\n", extracted_text) # Display extracted text (optional)

docs = extracted_text.split("\n") # Split by lines

doc_embeddings = model.encode(docs)

# Display the embeddings
print("Embeddings shape:", doc_embeddings.shape)
print("Embeddings:\n", doc_embeddings)

```

```

from chromadb import Client
from chromadb.config import Settings

client = Client(Settings(persist_directory="db/"))

collection2 = client.get_or_create_collection(name="Students2")

# Add your documents and embeddings
for i, doc in enumerate(docs):
    collection2.add(
        documents=[doc],
        embeddings=[doc_embeddings[i].tolist()],
        ids=[f"doc_{i}"]
    )

# Retrieve stored data
stored_data = collection2.get() # Fetches all stored documents and embeddings

# Display stored documents and embeddings
print("Stored Documents:", stored_data['documents'])
print("Stored Embeddings:", stored_data['embeddings'])
print("Stored IDs:", stored_data['ids'])

```

Topics Covered:

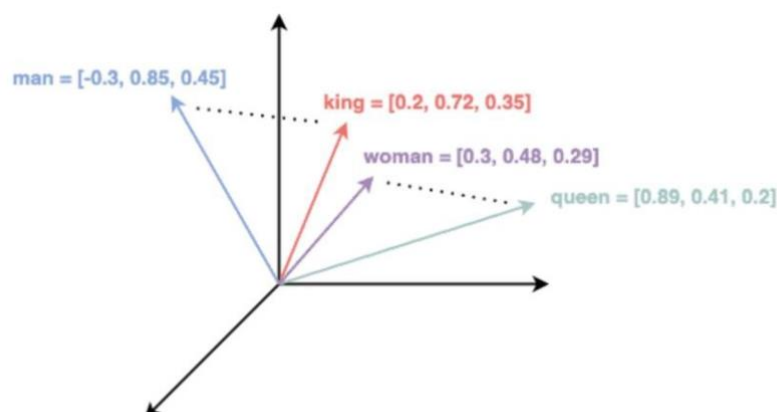
- Understanding Embeddings dimensions.
- Introduction to chunk size and overlap in vector databases.
- RAG using LangChain, Ollama and Hugging Face.

1. Understanding Embedding dimensions

In general, the dimensionality of word embedding refers to the number of dimensions in which the vector representation of a word is defined. This is typically a fixed value determined while creating the word embedding. The dimensionality of the word embedding represents the total number of features that are encoded in the vector representation.

Embeddings typically exist in high-dimensional spaces, often with hundreds of dimensions. In these spaces, the relative positions and angles between vectors are more crucial than the specific values of each dimension. The relationships between words or data points are encoded by their relative positions within this multidimensional landscape.

The nature of these dimensions is heavily influenced by the training data and the specific task for which the embedding model was trained. Embeddings trained on scientific texts may capture different nuances than those trained on general web text, leading to variations in the meaning of dimensions.



Word embeddings for “king”, “queen”, “man”, and “women” in a 3-dimensional space.

2. Introduction to chunk size and overlap in vector databases

Reference link: [Link 1](#)

While the choice of embedding model is an important consideration to ensure good quality retrieval for RAG, there is one key decision to be made before the embedding stage that can have a significant downstream impact — choosing the right chunking strategy for your data.

Chunking is the process of breaking down large pieces of text into smaller segments or chunks. In the context of RAG, embedding smaller chunks instead of entire documents to create the knowledge base means that given a user query, you only have to retrieve the most relevant document chunks, resulting in fewer input tokens and more targeted context for the LLM to work with.

The three key components of a chunking strategy are as follows:

- **Splitting technique**
- **Chunk size**
- **Chunk overlap**

3. RAG using LangChain, Ollama Llama3 and Hugging Face

- The previous code has been updated to enhance efficiency and minimize computational load. Instead of using the ***dunzhang/stella_en_1.5B_v5*** model, which has embedding dimensions of 8192, the ***bijaygurung/stella_en_400M_v5*** model with embedding dimensions of 1024 is now being used. This adjustment reduces resource consumption while maintaining effective performance.

LangChain: It is an open-source framework for building applications based on large language models (LLMs). LLMs are large deep-learning models pre-trained on large amounts of data that can generate responses to user queries—for example, answering questions or creating images from text-based prompts.

The following sections describe benefits of LangChain:

- Repurpose language models
- Simplify AI development
- Developer support Concepts of LangChain:
- **Chains** are the fundamental principle that holds various AI components in LangChain to provide context-aware responses. A chain is a series of automated actions from the user's query to the model's output.

- Chains are made of **links**. Each action that developers string together to form a chained sequence is called a link. With links, developers can divide complex tasks into multiple, smaller tasks.

```
from langchain_community.document_loaders import UnstructuredFileLoader
from langchain_text_splitters import CharacterTextSplitter
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_chroma import Chroma
from langchain_community.llms import Ollama
from langchain.chains import RetrievalQA

# Loading PDF File
loader = UnstructuredFileLoader("test1.pdf")
docs = loader.load()

# Splitting Text
text_splitter = CharacterTextSplitter(
    separator="\n",
    chunk_size=4000,
    chunk_overlap=200
)

texts = text_splitter.split_documents(docs)

# Embeddings
embeddings = HuggingFaceEmbeddings()

# Vector Database (Chroma)
db = Chroma.from_documents(texts, embeddings)

# Retrieval
llm = Ollama(model="llama3.1:8b")
chain = RetrievalQA.from_chain_type(
    llm,
    retriever=db.as_retriever()
)

question="Can you summarize the document?"
result = chain.invoke({"query": question})
print(result['result'])
```

The above code works on the HuggingFaceEmbeddings class from the langchain_huggingface module. This class is designed to generate embeddings using models from Hugging Face's transformers library. However, the following ways were explored:

- Using Embedding Models from Hugging Face: Models like `bijaygurung/stella_en_400M_v5` were found to be suitable for generating embeddings. However, to use these models locally, downloading the entire model is required, which may pose challenges in terms of storage and setup, especially for larger models.

- Using Embedding Models from Ollama: Ollama supports [embedding models](#), making it possible to build retrieval augmented generation (RAG) applications that combine text prompts with existing documents or other data. However, it was observed that the Ollama model (all-minilm) does not include an `embed_documents` method, which is essential for integration with ChromaDB or similar frameworks.

Topics Covered:

- Using Embedding Model locally with a Vector Database
- Integrating Chroma as the Vector Database.
- Challenges Encountered with loading LLM Locally

1. Using Embedding Model locally with a Vector Database

In the current setup, using the Hugging Face embedding model through API calls has been feasible. However, ensuring data security is crucial for any large-scale product, as it often involves handling sensitive information that cannot be exposed through API calls.

To address this concern, the *dunzhang/stella_en_400M_v5* model was selected from the [MTEB Leaderboard](#) for the embedding process. This model embeds input text locally into vectors, ensuring data privacy and security. The generated vectors are then stored in a vector database for efficient retrieval and processing.

Initially, the plan was to leverage the advantages that LangChain offers for developing Retrieval-Augmented Generation (RAG) applications and to utilize the *HuggingFaceEmbeddings* library for embedding tasks. However, despite extensive troubleshooting and consulting various forums, these efforts did not yield a solution. Ultimately, the model was successfully loaded locally using the Transformers library, which also provides dedicated *embed_documents* and *embed_query* functions to facilitate the embedding process.

Challenges Encountered:

- The embedding model requires a GPU for effective implementation. However, since macOS does not provide dedicated GPU support, the entire setup had to be migrated to a Windows system to ensure proper functionality and continued progress.
- The embedding model requires the implementation of CUDA (Compute Unified Devicen Architecture) along with the correct versions of the Torch, TorchVision, and TorchAudio libraries.
- However, the xformers library was only compatible with a specific version of Torch (v2.1.0), which did not align with the installed version of CUDA, resulting in compatibility issues.

```

# Using Embedding Model (Stella) directly

import os
import torch
from transformers import AutoModel, AutoTokenizer
from sklearn.preprocessing import normalize

model_dir = "D:/EY Internship/Stella"
vector_dim = 1024
vector_linear_directory = f"2_Dense_{vector_dim}"

model = AutoModel.from_pretrained(model_dir, trust_remote_code=True).cuda().eval()
tokenizer = AutoTokenizer.from_pretrained(model_dir, trust_remote_code=True)

vector_linear = torch.nn.Linear(in_features=model.config.hidden_size, out_features=vector_dim)

vector_linear_dict = {
    k.replace("linear.", ""): v for k, v in
    torch.load(os.path.join(model_dir, f"{vector_linear_directory}/pytorch_model.bin")).items()
}
vector_linear.load_state_dict(vector_linear_dict)
vector_linear.cuda()

def embed_documents(docs):
    with torch.no_grad():
        input_data = tokenizer(docs, padding="longest", truncation=True, max_length=512,
                                return_tensors="pt")
        input_data = {k: v.cuda() for k, v in input_data.items()}
        attention_mask = input_data["attention_mask"]
        last_hidden_state = model(**input_data)[0]
        last_hidden = last_hidden_state.masked_fill(~attention_mask[..., None].bool(), 0.0)
        docs_vectors = last_hidden.sum(dim=1) / attention_mask.sum(dim=1)[..., None]
        docs_vectors = normalize(vector_linear(docs_vectors).cpu().numpy(), axis=1)
    return docs_vectors

embeddings = embed_documents([doc.page_content for doc in texts])
print(embeddings)

```

Loading Stella model locally using Transformers library

2. Integrating Chroma DB as the Vector Database

ChromaDB is an open-source vector database designed to store and manage highdimensional vector embeddings generated from various data types, such as text or images. It enables efficient similarity search and retrieval, making it useful for applications in machine learning, natural language processing, and AI-driven tasks that require fast and scalable access to embedded data representations.

One of the challenges encountered was handling complex data types which took several revisions to work with the embedding model without problems.

```

# Option 2: Using Chroma Directly

from chromadb import Client

chroma_client = Client()
collection_name = "collection1"
db = chroma_client.get_or_create_collection(name=collection_name)

ids = [f"doc_{i}" for i in range(len(texts))]

db.add(
    documents=[doc.page_content for doc in texts],
    embeddings=[embedding.tolist() for embedding in embeddings],
    metadatas=[{"id": i} for i in range(len(texts))],
    ids=ids
)

Retrieval of text from database
results = db.get(
    ids=[f"doc_{i}" for i in range(len(texts))]
)

for i, result in enumerate(results["documents"]):
    print("Document ID:", results["metadatas"][i]["id"])
    print("Text:", result)
    print("-" * 50)

```

Using Chroma database to store embeddings and retrieve data

3. Challenges Encountered with loading LLM Locally

In the process of attempting to use Meta Llama 3.1 (8B) locally, several key observations have been made:

- **Model size:** Cloning Meta Llama 3.1 (8B) from Hugging Face's repository results in a substantial final file size of around 60 GB, despite the expected size for the 8B variant being approximately 15 GB.
- **Memory (RAM) limitation:** Running inference on a local Windows machine with 16 GB of RAM frequently results in kernel crashes.
- **Quantization Challenges:** Despite exploring quantization options like Auto-round, GPTQ, and BitsAndBytes, the 4-bit quantization has not yielded significant improvements in loading speed or inference time. Additionally, there were compatibility issues with BitsAndBytes and CUDA, although the following code functions as intended.

```

from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig
import torch

model_name = "meta-llama/Meta-Llama-3.1-8B-Instruct"

# Check for bfloat16 support
compute_dtype = torch.bfloat16 if torch.cuda.is_bf16_supported() else torch.float16

# Define 4-bit quantization configuration
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type='nf4',
    bnb_4bit_compute_dtype=compute_dtype,
    bnb_4bit_use_double_quant=True,
)

# Load the quantized model
model = AutoModelForCausalLM.from_pretrained(model_name, quantization_config=bnb_config)
tokenizer = AutoTokenizer.from_pretrained(model_name)

# Verify the model loads without issues
print("Model loaded successfully with 4-bit quantization.")

```

Performing 4-bit Quantization on Llama 3.1 (8b) via BitsAndBytes

- Using Ollama: The most common solution provided online for loading the LLM locally is to use Ollama. However, since Ollama was only recently released for Windows, its performance isn't as optimized as on Linux or macOS, leading to slower response times. Additionally, it's worth noting that the models provided by Ollama are already quantized.

The final project focuses on Document Intelligence for Generative AI, exploring two distinct methods: local and online implementation. By following the conventional Retrieval-Augmented Generation (RAG) approach, the project processes text by extracting it from the source, pre-processing, embedding, storing, and then querying it for various use cases.

This approach optimizes time and effort by using pre-trained models directly rather than training from scratch, which also allows for a more targeted, context-specific application of the language model (LLM), making it less generalized and more suited to specific needs. The project includes two files, each with its benefits and insights gained from running the implementation on both macOS and Windows.

A) App1.ipynb (local)

- Implemented on Windows using the locally loaded [thedunzhang/stella_en_400M_v5](#) model to embed input text and generate vector embeddings.
- Integration with Chroma DB and the retrieval (LLM-based) posed challenges, so the embedding process was completed using the *SentenceTransformerEmbeddings* library instead.

```
from sentence_transformers import SentenceTransformer
from langchain.embeddings.base import Embeddings

class SentenceTransformerEmbeddings(Embeddings):
    def __init__(self, model_name):
        self.model = SentenceTransformer(model_name, trust_remote_code=True)

    def embed_documents(self, texts):
        return self.model.encode(texts, convert_to_tensor=True).tolist()

    def embed_query(self, text):
        return self.model.encode(text, convert_to_tensor=True).tolist()

embeddings = SentenceTransformerEmbeddings("D:/EY Internship/Stella")
```

Usage of SentenceTransformerEmbeddings to load the embedding model locally

Key Findings

1. **Case 1: Load the local Stella model and run the pipeline**
GPU Used: Yes

Time Taken: 8.69 minutes

Memory (RAM) Usage: Approximately 900 MB left out of 16 GB (high usage)

Issue: Very slow solution for real-time use cases

2. Case 2: Running the pipeline without using loading Stella model

GPU Used: No (CPU only)

Time Taken: 3.62 minutes

Memory (RAM) Usage: Approximately 1.5 – 2 GB left out of 16 GB (less usage than Case 1)

Issue: Faster solution than Case 1 but still slow for real-time use cases

3. Case 3: Running the pipeline using Groq (Fast Inference)

GPU Used: No (API Based)

Time Taken: 2.15 seconds

Memory (RAM) Usage: None (API Based)

Issue: Fastest solution, it does not ensure complete isolation and security of sensitive data.

- Cannot be used on MacOS platform, as the embedding model requires CUDA to run, and macOS only has dedicated GPUs, which do not support CUDA.

B) App2.ipynb (Online)

- Leverages the benefits of high-performance text generation and embedding, with built-in performance and ease of implementation using tools such as Hugging Face, LangChain, and others.
- This is because the computation is performed off-device, making it feasible and cost-effective. However, it does not guarantee security, as the data is sent to the cloud for processing.

```
from sentence_transformers import SentenceTransformer
from langchain.embeddings.base import Embeddings

class SentenceTransformerEmbeddings(Embeddings):
    def __init__(self, model_name):
        self.model = SentenceTransformer(model_name, trust_remote_code=True)

    def embed_documents(self, texts):
        return self.model.encode(texts, convert_to_tensor=True).tolist()

    def embed_query(self, text):
        return self.model.encode(text, convert_to_tensor=True).tolist()

embeddings = SentenceTransformerEmbeddings("bijaygurung/stella_en_400M_v5")
```

Usage of SentenceTransformerEmbeddings to load the embedding model remotely

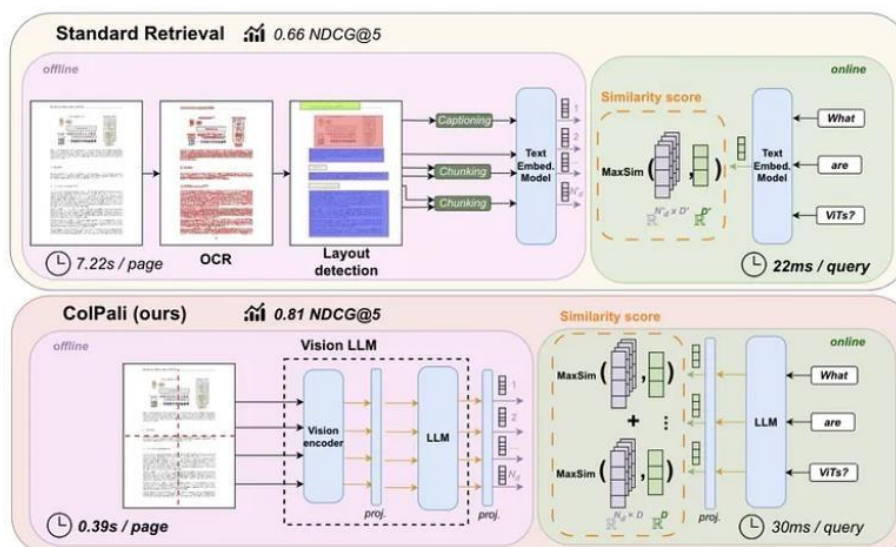
- Can be implemented on both Windows and MacOS platform, making it more flexible. In most cases, the retrieval section (Ollama) has been faster in MacOS due to its optimization as compared to Windows (Note: Ollama has recently been released for Windows platform).

Challenges Encountered

- **Hardware Constraints:** Large Language Models (LLMs) are typically substantial in size and demand significant computational power and memory resources. Running these models on local hardware posed considerable challenges due to limited resources.
- **Library Dependencies:** Ensuring compatibility among libraries, including specific versions and interdependencies, introduced additional complexity and required significant time to resolve compatibility issues effectively.

Topics Explored

- **Groq:** [Groq](#) is a high-performance AI platform designed for fast inference in both cloud environments and on-premises AI compute centers. It provides specialized architecture to handle intensive AI workloads, enabling faster processing times and efficient use of resources for large-scale model inference.
- **ColPali:** Traditional RAG (Retrieval Augmented Generation) Tools are useful but often challenging, the whole embedding process can be slow and tends to propagate errors. [ColPali](#) (ColBERT + PaliGemma) comes in. Instead of isolating text, ColPali embeds document pages as images using the PaliGemma model, breaking down the page into patches. With ColPali, we cut indexing times significantly without sacrificing performance.



Area for Improvement

- Lowering the *embedding dimension* can improve inference speed, say from default 1024 to 512 or even 256. The choice of *thedunzhang/stella_en_400M_v5* was because of its flexibility in dimension options, ranging from 246, 512, 768, 1024, 2048, 4096, 6144, up to 8192.